

DAY 14 – HASHING PROBLEMS & OPTIMIZATION

Mastery of DSA in 50 Days

February 2026

1 Today's Focus

- Solving real interview problems using hashing
- Optimizing from brute force to $O(n)$ time
- Understanding time–space tradeoff
- Avoiding nested loops with smart lookups
- Strengthening pattern recognition for hashing

2 Why Hashing is Important in Problems?

Hashing shines when we need:

- Fast existence check ($O(1)$ average)
- Fast frequency tracking
- Duplicate detection
- Pair / group finding without $O(n^2)$

It trades $O(n)$ space for massive time improvement — very common trade-off in interviews.

3 Problem 1 – Pair with Given Sum (Unsorted Array)

Problem Statement: Given an unsorted array of integers and a target sum, determine if there exist two numbers that add up to the target.

Example:

Input: `arr = [8, 4, 1, 6]`, `target = 10`

Output: `True` (`4 + 6 = 10`)

3.1 Naive Approach (Brute Force)

Nested loops — check every pair.

Time Complexity: **$O(n^2)$** Space Complexity: **$O(1)$**

3.2 Hashing Approach – Optimal

Use a set to store seen elements.

```
1 def has_pair_with_sum(arr, target):
2     seen = set()
3
4     for num in arr:
5         complement = target - num
6         if complement in seen:
```

```

7         return True
8         seen.add(num)
9
10        return False
11
12    # Test
13    arr = [8, 4, 1, 6]
14    print(has_pair_with_sum(arr, 10))    # True
15    print(has_pair_with_sum(arr, 15))    # False

```

Time Complexity: $O(n)$ Space Complexity: $O(n)$

4 Problem 2 – Count Distinct Elements

Problem Statement: Count the number of unique (distinct) elements in an array.

Example:

Input: [1, 2, 2, 3, 4, 4]

Output: 4

Hashing Logic (using set):

```

1 def count_distinct(arr):
2     unique = set(arr)
3     return len(unique)
4
5 # OR manual
6 def count_distinct_manual(arr):
7     seen = {}
8     for num in arr:
9         seen[num] = 1 # or any value
10    return len(seen)
11
12 # Test
13 arr = [1, 2, 2, 3, 4, 4]
14 print(count_distinct(arr))    # 4

```

Time Complexity: $O(n)$ Space Complexity: $O(k)$ where k = distinct elements

5 Problem 3 – First Repeating Element

Problem Statement: Find the first element (in order of appearance) that repeats in the array. Return -1 if no repeating element exists.

Example:

Input: [10, 5, 3, 4, 3, 5, 6]

Output: 3 (3 appears second time first)

Hashing Logic (track seen elements):

```

1 def first_repeating(arr):
2     seen = set()
3
4     for num in arr:
5         if num in seen:
6             return num
7         seen.add(num)
8

```

```

9     return -1
10
11 # Test
12 arr = [10, 5, 3, 4, 3, 5, 6]
13 print(first_repeating(arr))    # 3

```

Time Complexity: $O(n)$ Space Complexity: $O(n)$

Alternative (if order of first occurrence matters): Use dict to store first index.

6 Hashing vs Sorting Comparison

Approach	Time Complexity	Space Complexity	Preserves Order?
Sorting	$O(n \log n)$	$O(1)$ or $O(n)$	No
Hashing	$O(n)$	$O(n)$	Yes

Table 1: Hashing vs Sorting for Frequency/Duplicates

Choose hashing when:

- Linear time is needed
- Order matters
- Extra space is allowed

7 Key Learnings – Day 14

- Hashing turns many $O(n^2)$ problems into $O(n)$
- Set $\rightarrow$ fast existence & duplicate check
- Dict / Counter $\rightarrow$ frequency & mapping
- Time–space tradeoff: extra $O(n)$ space for huge time gain
- Pattern: “lookup needed?” $\rightarrow$ think hash table

8 Interview Tips

- Always present brute force first $\rightarrow$ then optimize with hashing
- Say: “Using a hash set gives us $O(n)$ time instead of $O(n^2)$ ”
- Ask constraints: “Are negative numbers allowed? Duplicates? Range of values?”
- Mention: “In worst case (all collisions), $O(n^2)$ — but average $O(n)$ ”
- Handle edge cases: empty array, single element, all unique

9 Note to Learners

Hashing is the **backbone** of modern problem-solving.

You will use it in:

- Almost every medium/hard array/string problem
- Two Sum, 3Sum, group anagrams
- Subarray sum equals k
- Longest substring without repeating characters
- Most FAANG interview questions

Master it — it unlocks hundreds of problems.

End of Day 14

Next: Day 15 – Recursion Basics